

- ▼ 机器学习任务分类
 - ▼ 监督学习
 - 定义
 - 两大任务
 - ▼ 无监督学习
 - 定义
 - 三大任务
- ▼ 算法与模型
 - ▼ 线性回归
 - Notation
 - ▼ 代价函数
 -  均方误差 (Mean Squared Error, MSE)
 -  代价函数的可视化
 - ▼ Gradient Descent
 -  算法步骤
 - 核心更新规则:
 -  关键参数: 学习率 (α)
 -  Batch Gradient Descent
 -  代码片段
 -  正规方程
- ▼ 多类特征
 - ▼ 向量化
 - Notation
 -  代码片段
 - Gradient Descent 的多特征形式:
 - ▼ 特征缩放 (Feature Scaling)
 - 特征缩放方法:
 - ▼ Gradient Descent 收敛性
 - 学习曲线
- ▼ 逻辑回归
 - 为什么不是线性回归?
 - Sigmoid 函数
 - 决策边界 (Decision Boundary)
 - 代价函数
 - Gradient Descent
- ▼ 过拟合与正则化
 - ▼ 欠拟合 (Underfitting) 与 过拟合 (Overfitting)
 - 欠拟合 (Underfitting / High Bias):
 - 过拟合 (Overfitting / High Variance):
 - ▼ 正则化 (Regularization)
 - L2 正则化 (L2 Regularization / Ridge Regression):
 - L1 正则化 (L1 Regularization / Lasso Regression):
 - 偏差 (Bias) 与 方差 (Variance) 的权衡

机器学习任务分类

监督学习

定义

监督学习是从带有 Tag [\[1\]](#) (或称 Right Answer) 的 Training Set 中学习得到一个 Func , 从而能够根据新的、未见过的 data 预测输出。

两大任务

- 回归 (Regression): 预测连续值的输出，例如房价、股票价格、温度等。
- 分类 (Classification): 预测离散值的输出，例如判断邮件是否为垃圾邮件（是/否）、肿瘤是良性还是恶性、图片中是猫还是狗（猫/狗/其他）。

无监督学习

定义

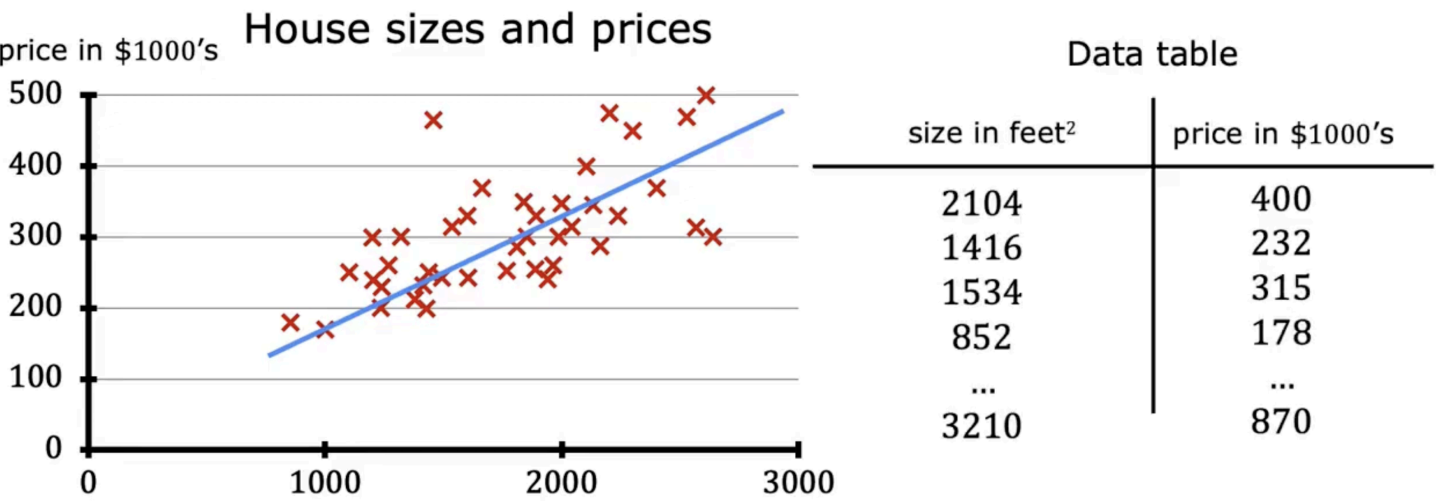
无监督学习是在无标签的训练集中寻找 data 之间的内在结构或分布。

三大任务

- 聚类 (Clustering): 将训练集中的样本分成若干个组（簇），使得同一组内的样本彼此相似，而不同组的样本差异较大。
- 降维 (Dimensionality Reduction): 在保留 data 核心信息的前提下，减少 data 特征的量。
- 异常检测 (Anomaly Detection): 识别不符合预期模式或行为的 Data Point、事件或观测值。

算法与模型

线性回归



Notation

Notation	Description
x	input variable, feature
y	output variable (target)
m	number of training examples
(x, y)	single training example
$(x^{(i)}, y^{(i)})$	i^{th} training example
$f(x) = wx + b$	function, model
w, b	parameters
$\hat{y}$	estimate for y /prediction

代价函数

🎯 均方误差 (Mean Squared Error, MSE)

- 对于训练样本 i ($x^{(i)}, y^{(i)}$)，模型的预测值是 $\hat{y}^{(i)} = f(x^{(i)}) = wx^{(i)} + b$ 。

那么，整个训练集上的均方误差代价函数 $J(w, b)$ 为：

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

或者，将 $\hat{y}^{(i)}$ 改为 $f(x^{(i)})$ ：

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})^2$$

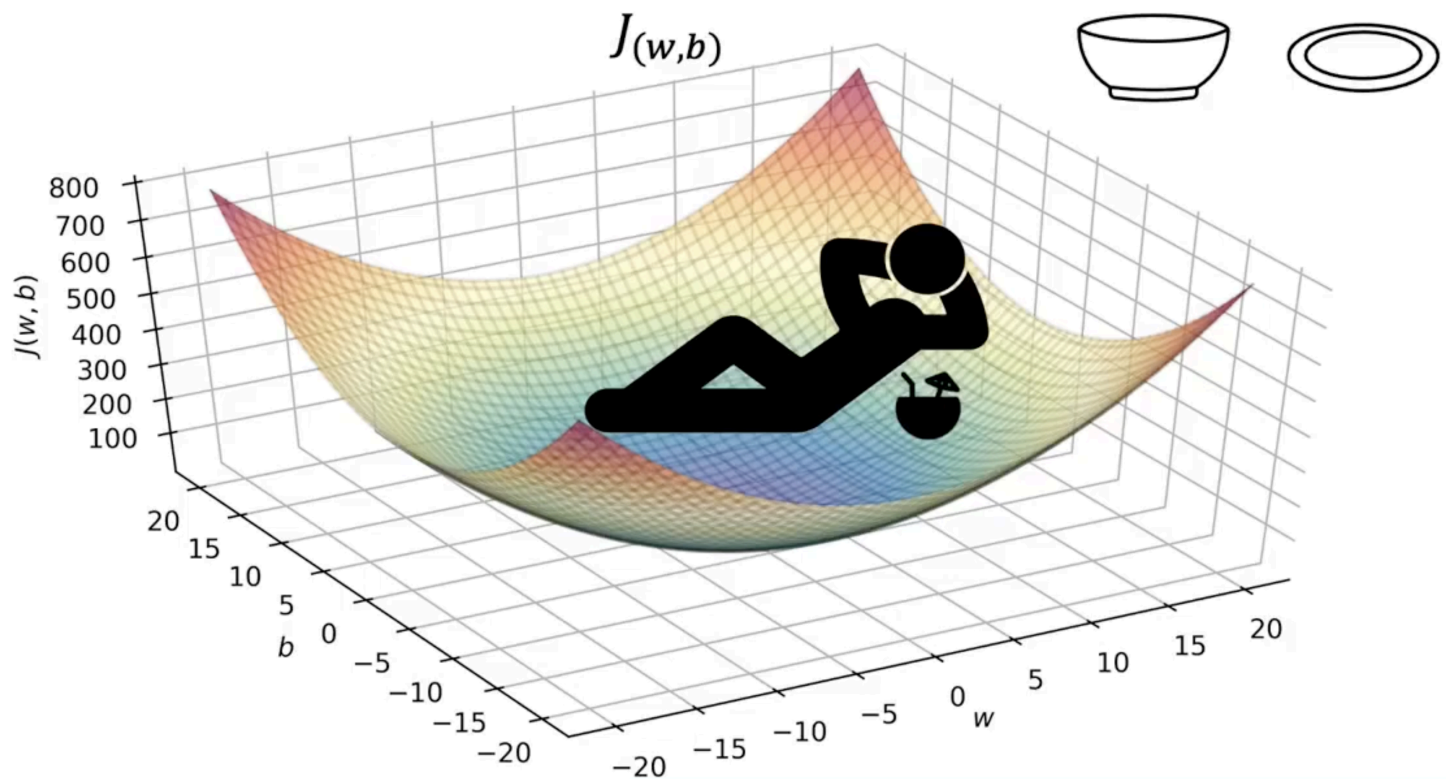
目标即找到一对参数 (w, b) ，使得代价函数 $J(w, b)$ 的值最小。

$$\min_{w, b} J(w, b)$$

通过不断修正 w 和 b ，直到 $J(w, b)$ 达到最小值，此时的模型 $f(x) = wx + b$ 就是拟合训练集好直线。

🇮🇹 代价函数的可视化

只有 w 一个参数 (即 $b = 0$ 时)。 $J(w)$ 会是一个开口向上的抛物线，它的最低点即为最佳 w 值。



当有 w 和 b 两个参数时， $J(w, b)$ 就是一个三维空间中的碗状曲面，最低点就是最佳的 (w, b) 组合。

Gradient Descent

你站在一座大山的山顶，旁边一片蒙蒙，你无法看清山底在哪里。但你知道，如果每次都朝着最陡峭的下坡方向走一小步，最终就能走到山谷的最低点。

- 当前位置：当前参数 w 和 b 。
- 山的高度：代价函数 $J(w, b)$ 的值。
- 最陡峭的下坡方向：代价函数对 w 和 b 的 gradient 的反方向。
- 一小步：参数修正的步长，由学习率 (α) 控制。

🍷 算法步骤

1. 初始化参数：随机选择一个初始的 w 和 b 值（一般初始化为0）。

2. 迭代以下步骤直到收敛：

- 求得 **gradient**：求得代价函数 $J(w, b)$ 关于 w 和 b 的偏导数。
- 修正参数：沿着 **gradient** 的反方向，按照学习率 (α) 指定的步长更新 w 和 b 。

核心更新规则：

对于参数 w ：

$$w := w - \alpha \frac{\partial}{\partial w} J(w, b)$$

对于参数 b ：

$$b := b - \alpha \frac{\partial}{\partial b} J(w, b)$$

注意： w 和 b 必须同步更新，而不是先更新 w 再用新的 w 来更新 b 。

🔧 关键参数：学习率 (α)

学习率 α 是 Gradient Descent 中一个非常核心的超参数，它决定了每次参数更新的“步子”迈多大。

- α 太小：Gradient Descent 会非常缓慢地收敛，需很多次迭代才能到达最小值。
- α 太大：Gradient Descent 可能会越过最小值，甚至发散，永远无法收敛。

对于线性回归的代价函数来说：

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})^2$$

- 对 w 的偏导：

$$\frac{\partial}{\partial w} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x^{(i)}$$

- 对 b 的偏导：

$$\frac{\partial}{\partial b} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})$$

所以，线性回归的参数更新规则为：

$$w := w - \alpha \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b := b - \alpha \frac{1}{m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})$$

🌈 Batch Gradient Descent

Batch Gradient Descent 的特点是：

- 在每次参数更新时，都会遍历所有 m 个训练样本来求得 **gradient**。
- 对于小型训练集，它通常能找到全局最优解（尤其对于凸函数而言）。
- 对于大型数据集，每次迭代计算任务会很大，效率较低。

代码片段

```
w = 0.0
b = 0.0
learning_rate = 0.01 # alpha
num_iterations = 1000 # the number of iterations

x = [x_1, x_2, ..., x_m]
y = [y_1, y_2, ..., y_m]
m = len(x)

for i in range(num_iterations):
    # f_x = w * x + b
    f_x = [w * x_i + b for x_i in x]

    # calculate the gradient of w
    dw = (1/m) * sum([(f_x[j] - y[j]) * x[j] for j in range(m)])

    # calculate the gradient of b
    db = (1/m) * sum([(f_x[j] - y[j]) for j in range(m)])

    # update the w and b synchronously
    w = w - learning_rate * dw
    b = b - learning_rate * db

    # print J(w,b)
    # if i % 100 == 0:
    #     cost = (1/(2*m)) * sum([(f_x[j] - y[j])**2 for j in range(m)])
    #     print(f"Iteration {i}, Cost: {cost:.4f}")

print(f"优化得: w = {w:.2f}, b = {b:.2f}")
```

正规方程

正规方程，得先将训练集改成矩阵形式。

- 特征矩阵 X : 一个 $m \times (n + 1)$ 维的矩阵。每一行代表一个训练样本，每一列代表一个特征（包括常特征，例如 $x_0 = 1$ ）。

$$X = \begin{bmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} = \begin{bmatrix} x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ x_0^{(2)} & x_1^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{bmatrix}$$

- 目标值向量 y : 一个 $m \times 1$ 维的列向量，包含所有训练样本的真实输出值。

$$y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

有了这些矩阵和向量，最优的参数 w 可以通过以下公式直接得到：

$$w = (X^T X)^{-1} X^T y$$

其中：

- X^T : 矩阵 X 的转置。
- $X^T X$: 矩阵 X 乘以其转置，得到一个 $(n + 1) \times (n + 1)$ 的方阵。
- $(X^T X)^{-1}$: $(X^T X)$ 的逆矩阵，核心，也是其局限性所在——只适用于线性回归以及训练集小的情况。

多类特征

向量化

Notation

Notation	Description
x_j	j^{th} feature
n	number of features
m	number of training examples
$\vec{x}^{(i)}$	features of i^{th} training example
$x_j^{(i)}$	value of feature j in i^{th} training example
$f(x) = \sum_{j=1}^n x_j w_j + b = \vec{x} \cdot \vec{w} + b$	function, model
$\hat{y}$	estimate for y /prediction

代码片段

```
import numpy as np

w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])

f = np.dot(w, x) + b
```

Gradient Descent 的多特征形式:

多元线性回归的 Gradient Descent 的更新规则对于每个参数 w_j 和 b 均是类似的:

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$
$$b := b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

代入矢量 $\vec{x}$ 得:

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$
$$b := b - \alpha \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)})$$

对 $j = 0, 1, \dots, n$ 同时进行更新。

特征缩放 (Feature Scaling)

当你有了多个特征时，它们的值范围可能差异巨大，如果不对特征进行缩放，可能会导致：

- Gradient Descent 收敛缓慢： 代价函数的等高线会变得非常狭长， Gradient Descent 的路径会非常曲折，难以快速找到最小值。
- 某些特征主导优化： 范围大的特征会在计算 gradient 时占据主导地位，使得模型对范围小的特征不够敏感。

特征缩放方法：

- 归一化 (Normalization)： 将特征值缩放到 $[0, 1]$ 之间。

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- 均值归一化 (Mean Normalization): 将特征值缩放到围绕 0 左右的范围，通常是 $[-1, 1]$ 。

$$x_{scaled} = \frac{x - \mu}{x_{max} - x_{min}}$$

- Z-score 标准化 (Standardization): 最常用，将特征值转换为均值为 0，标准差为 1 的分布。

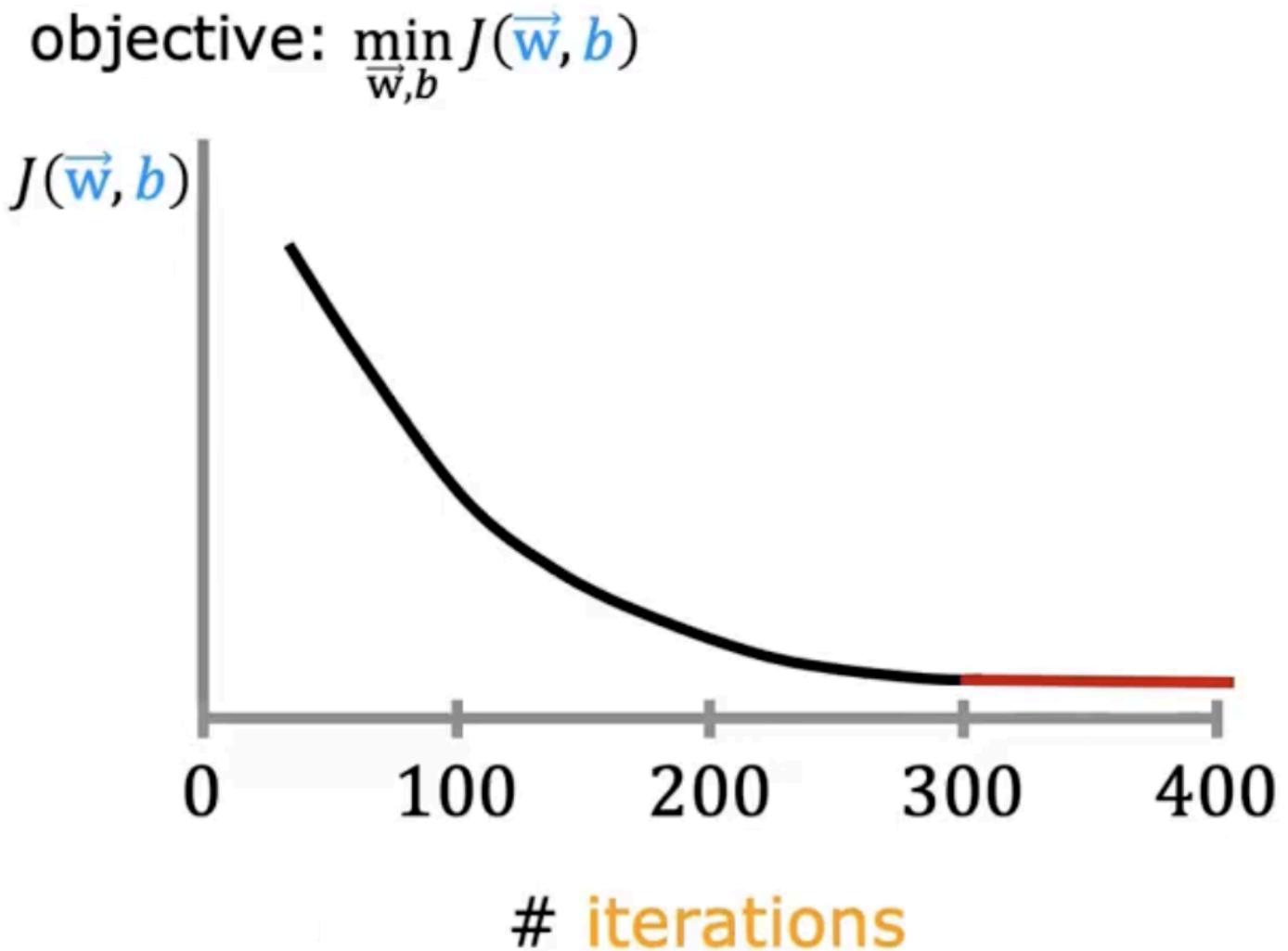
$$x_{scaled} = \frac{x - \mu}{\sigma}$$

其中 μ 是特征的平均值， σ 是特征的标准差。

Gradient Descent 收敛性

学习曲线

学习曲线中，代价函数的值随迭代进行而不断减小，最终收敛。



逻辑回归

为什么不是线性回归?

对于分类问题，比如预测肿瘤是“良性”（0）还是“恶性”（1），如果采取线性回归方法，模型的输出 $f(x) = w^T x + b$ 可能是一个任意的连续值，不太适合直接作为类别的概率。

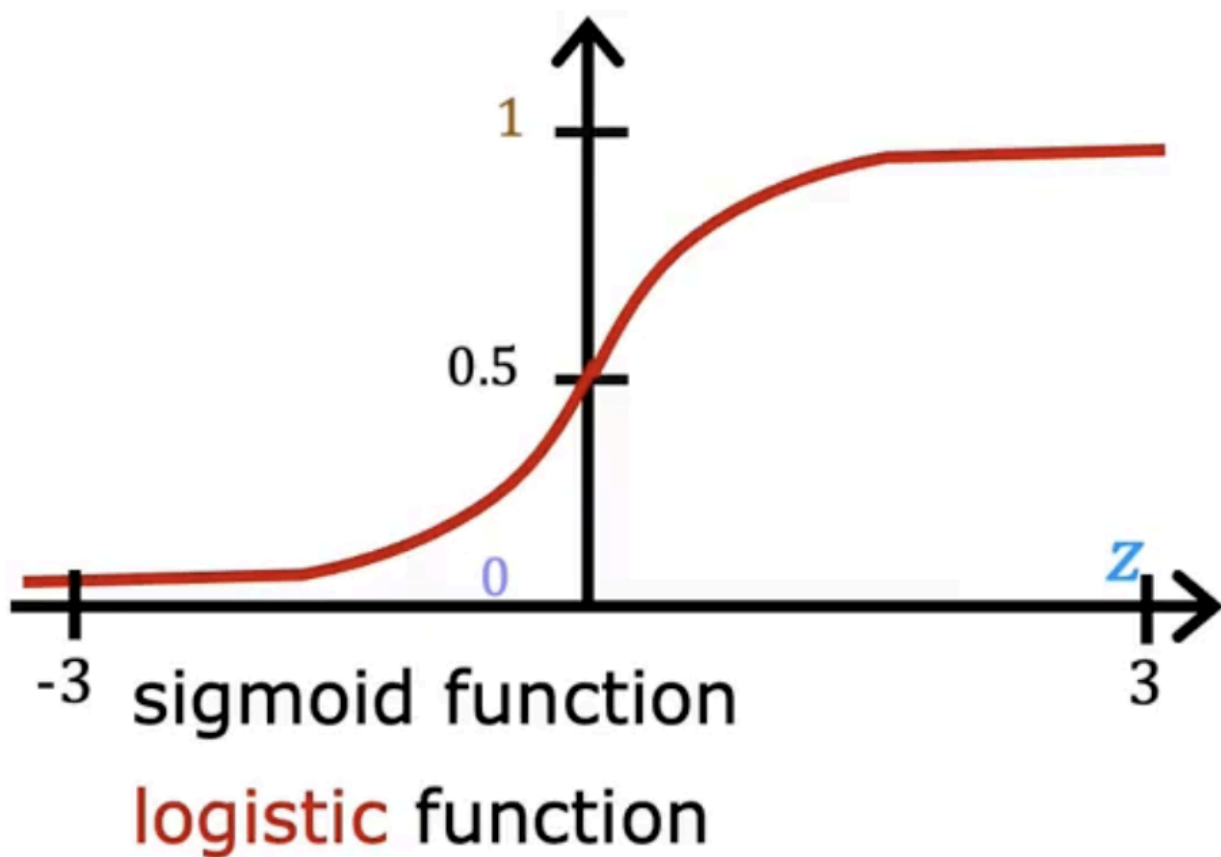
此外，如果将线性回归的输出通过阈值（比如 0.5）进行分类，那么异常值可能会严重扭曲直线的拟合，导致分类效果很差。

Sigmoid 函数

Sigmoid 函数（也称为 **Logistic 函数**），它能将任意实数值映射到 0 到 1 之间，使其可以解释为概率。

- 定义:

$$g(z) = \frac{1}{1 + e^{-z}}, 0 < g(z) < 1$$



- 模型假设函数 (Hypothesis Function):

将线性回归的输出 $w^T x + b$ 作为 z 输入到 Sigmoid 函数中:

$$f(x) = g(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

这个 $f(x)$ 的输出为给定输入 x 时, 属于正类别 (例如, “恶性”) 的概率, 即 $P(y = 1|x; w, b)$ 。

- 如果 $f(x)$ 接近 1, 即为正类别的概率高。
- 如果 $f(x)$ 接近 0, 即为负类别的概率高。

决策边界 (Decision Boundary)

- 决策规则:

- 如果 $f(x) \geq 0.5$, 预测 $\hat{y} = 1$ (正类别)。
- 如果 $f(x) < 0.5$, 预测 $\hat{y} = 0$ (负类别)。

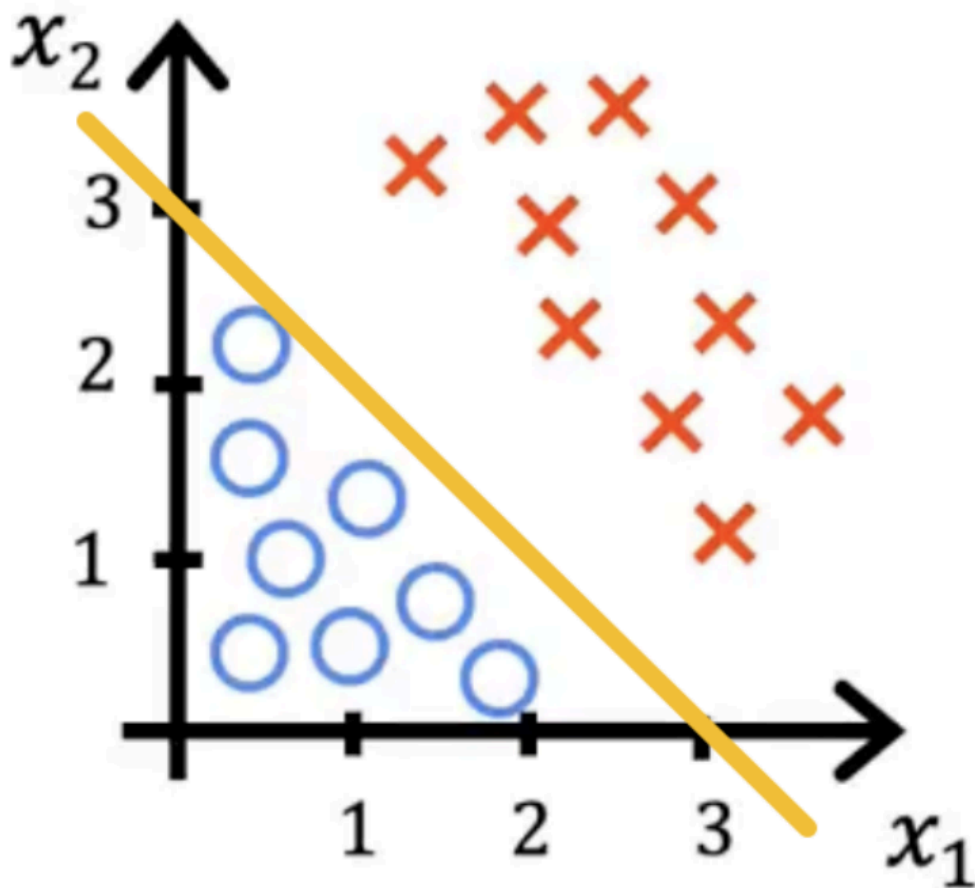
- 为什么是 0.5?

- 当 $g(z) \geq 0.5$ 时, 意味着 $z \geq 0$ 。
- 所以, 当 $w^T x + b \geq 0$ 时, 模型预测 $\hat{y} = 1$ 。
- 当 $w^T x + b < 0$ 时, 模型预测 $\hat{y} = 0$ 。

- 决策边界:

决策边界是特征空间中将不同类别区分开来的“线” (或在高维空间中的“超平面”), 其由方程 $w^T x + b = 0$ 定义。

Decision Boundary 可以是线性的, 就像 $w^T x + b = 0$, 如下图中 $g(z) = g(w_1 x_1 + w_2 x_2 + b)$, $w_1 = 1, w_2 = 1, b = -3$, 则 Decision Boundary 为 $z = x_1 + x_2 - 3$



也可以是非线性的，即在 x 中包含多项式特征，例如 $x_1^2 + x_2^2 = C$ ，那么其为一个圆。

代价函数

如今不能再使用线性回归的 MSE 作为代价函数。原因在于，将 $f(x)$ 代入 MSE 后，得到的 $J(\vec{w}, b)$ 是一个非凸函数，会有很多局部最小值，导致 Gradient Descent 可能无法找到全局最优解。

因此，可以采用对数损失函数 (Log Loss)，也称为交叉熵损失 (Cross-Entropy Loss)，它是一个凸函数，保证了 Gradient Descent 能够找到全局最优解。

首先，从代价函数中抽象出损失函数 L ，对于不同的模型只用选择不同的损失函数即可，例如在线性回归中损失函数选择的即为 $L = \frac{1}{2}(f(\vec{x}^{(i)}) - y^{(i)})^2$ ，即

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)}) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f(\vec{x}^{(i)}) - y^{(i)})^2$$

而本节的对数损失函数如下：

$$L(f(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

即

- 当 $y = 1$ ：代价是 $-\log(f(x))$ ，当 $f(x)$ 接近 1 时代价小，接近 0 时代价大。
- 当 $y = 0$ ：代价是 $-\log(1 - f(x))$ 。当 $f(x)$ 接近 0 时代价小，接近 1 时代价大。

最终的代价函数 $J(w, b)$ ：

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f(\vec{x}^{(i)}))]$$

Gradient Descent

首先，回顾一下参数的更新规则：

$$\begin{cases} w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \\ b := b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \end{cases}$$

为此得到

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

参数 b 类似，故可知

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b := b - \alpha \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)})$$

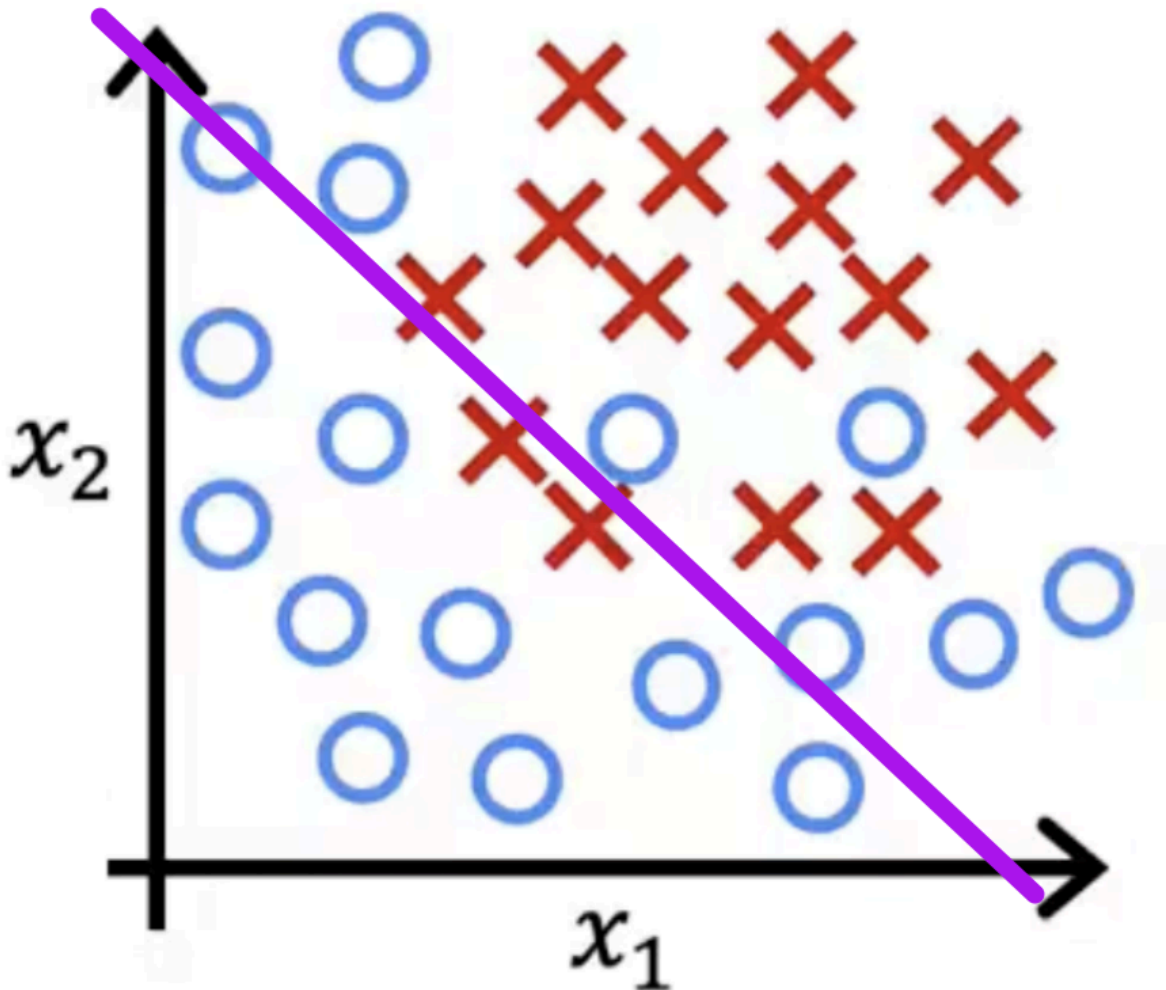
形式上，此处的更新规则与线性回归处的规则完全一样，然而这里的 $f(\vec{x}^{(i)})$ 是逻辑回归的假设函数，即 $f(\vec{x}^{(i)}) = \frac{1}{1+e^{-(w^T \vec{x}^{(i)} + b)}}$ 。

过拟合与正则化

欠拟合 (Underfitting) 与 过拟合 (Overfitting)

欠拟合 (Underfitting / High Bias):

由于模型过于简单，无法捕捉 data 中的基本规律。在训练集和测试集上的表现都很差。

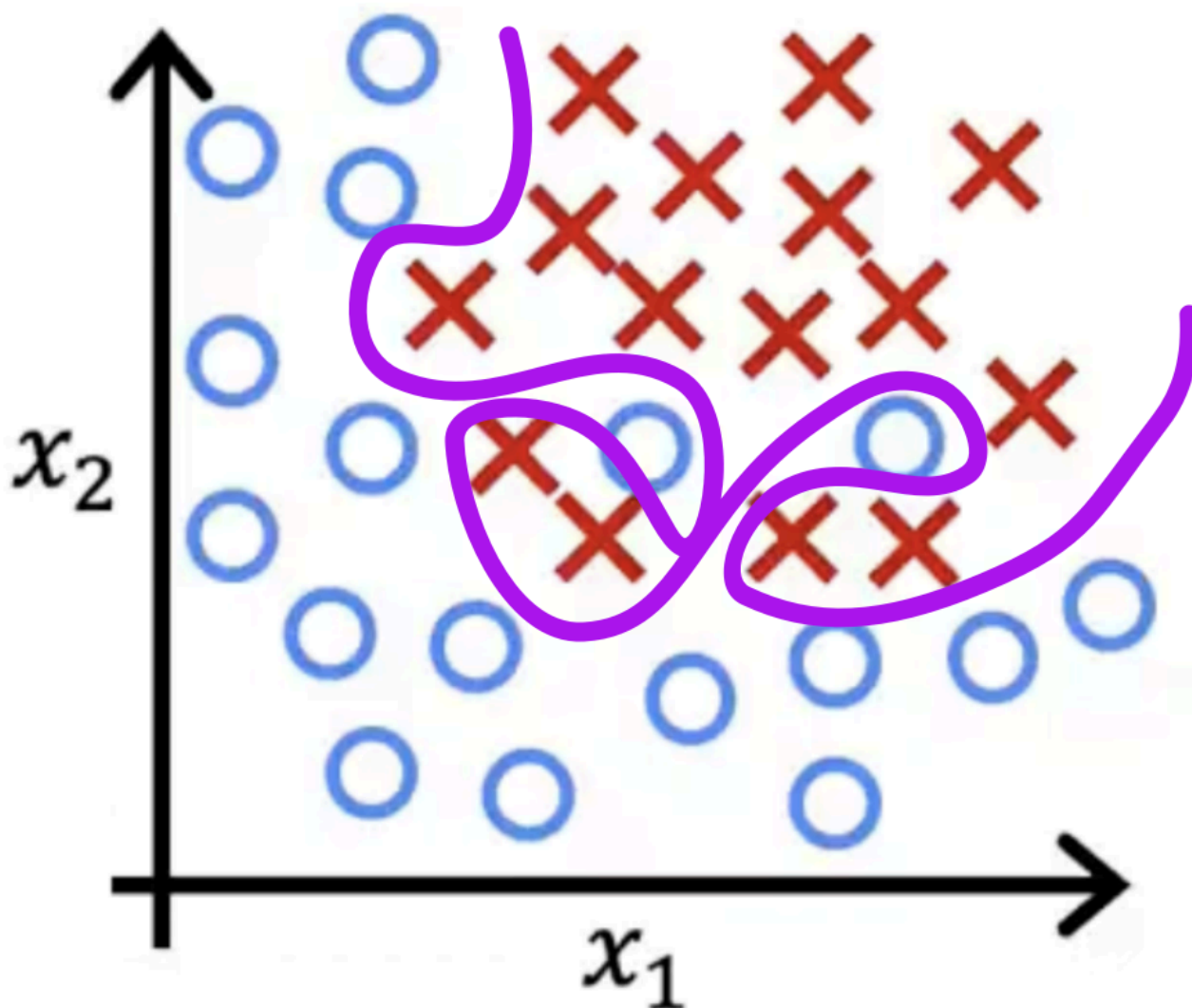


- 原因：模型不够精妙（例如，用直线拟合曲线）、特征太少、训练时间不够。
- 解决：
 - 增加特征（多项式特征、组合特征）。

- 增加模型复杂性（多项式回归）。

过拟合 (Overfitting / High Variance):

由于模型过于复杂，过分学习了训练集中的噪声和细节，导致在训练集上表现非常好，但在测试集上表现很差（泛化能力差）。



- 原因：模型过于复杂（例如，用高次多项式拟合小训练集）、训练集太小、特征太多。
- 解决：
 - 增大训练集。
 - 减少特征。
 - 正则化。
 - 简化模型。

正则化 (Regularization)

正则化在最小化代价函数的同时，惩罚模型参数的复杂度。通过对参数 w （通常不包括截距项 b ）的大小施加限制，可以使模型简单化，从而提高其泛化能力。

L2 正则化 (L2 Regularization / Ridge Regression):

- 惩罚项：添加所有参数平方和的惩罚项。
- 代价函数：
 - 线性回归：

$$J(\vec{w}, b) = \left[\frac{1}{2m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)})^2 \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

- 逻辑回归：

$$J(\vec{w}, b) = \left[-\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f(\vec{x}^{(i)}))] \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

- 效果：使得参数 w_j 趋向于较小但不为 0 的值。
- *Gradient Descent*： w_j 更新会多出一个衰减项。

$$w_j := w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{1}{m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

L1 正则化 (L1 Regularization / Lasso Regression):

- 代价函数（线性回归）：

$$J(\vec{w}, b) = \left[\frac{1}{2m} \sum_{i=1}^m (f(\vec{x}^{(i)}) - y^{(i)})^2 \right] + \frac{\lambda}{2m} \sum_{j=1}^n |w_j|$$

偏差 (Bias) 与 方差 (Variance) 的权衡

- 偏差 (**Bias**)：高偏差对应欠拟合，模型预测与真实值之间存在系统性差异。
- 方差 (**Variance**)：高方差对应过拟合，模型对训练集的具体样本过于敏感。

1. 标准术语集

Tag: 标签

Training Set: 训练集

Func: 函数

data: 数据

Data Point: 数据点

Gradient Descent: 梯度下降

